Special Participation B - Gemini 3 on HW2

Executive Summary

The LLM-generated code demonstrates a high level of accuracy and adherence to the assignment requirements. In almost all cases, the logic implemented by the LLM is identical or functionally equivalent to the staff solutions. The code style is consistent with the provided codebase, and the implementations are generally concise and idiomatic.

Key observations:

- **Correctness**: The core algorithms (optimizers, neural network layers, backpropagation) are implemented correctly.
- **Style**: The code follows standard Python and NumPy practices. Variable naming is consistent with the surrounding code.
- **Differences**: Minor differences exist in hyperparameter choices (e.g., learning rate for the best model) and some implementation details (e.g., using `np.zeros` vs `np.random.normal(scale=0)`), but these do not affect correctness.
- **Completeness**: All identified TODOs in the provided files were addressed.

TODO by TODO Evaluation

`deeplearning/optim.py`

`sgd_momentum`

LLM Implementation:

```
v = config['momentum'] * v + dw
next_w = w - config['learning_rate'] * v
```

Staff Solution:

```
v = config['momentum'] * v + dw
next_w = w - config['learning_rate'] * v
```

Comments:

- **Correctness**: Identical.
- **Style**: Clear and concise.

`rmsprop`

LLM Implementation:

```
config['cache'] = config['decay_rate'] * config['cache'] + (1 - config['decay_rate']) *
next_w = w - config['learning_rate'] * dw / (np.sqrt(config['cache']) + config['epsilon
```

Staff Solution:

```
cache = config['decay_rate'] * config['cache'] + (1 - config['decay_rate']) * (dw ** 2)
next_w = w - config['learning_rate'] * dw / (np.sqrt(cache) + config['epsilon'])
config['cache'] = cache
```

Comments:

- **Correctness**: Functionally equivalent.
- **Style**: LLM updates the dictionary directly, while staff solution uses a local variable first. Both are acceptable.

`adam`

LLM Implementation:

```
config['t'] += 1
config['m'] = config['beta1'] * config['m'] + (1 - config['beta1']) * dw
config['v'] = config['beta2'] * config['v'] + (1 - config['beta2']) * (dw**2)
m_hat = config['m'] / (1 - config['beta1']**config['t'])
v_hat = config['v'] / (1 - config['beta2']**config['t'])
next_w = w - config['learning_rate'] * m_hat / (np.sqrt(v_hat) + config['epsilon'])
```

Staff Solution:

```
config['t'] += 1
m = config['beta1'] * config['m'] + (1 - config['beta1']) * dw
v = config['beta2'] * config['v'] + (1 - config['beta2']) * (dw ** 2)
mt = m / (1 - config['beta1'] ** config['t'])
vt = v / (1 - config['beta2'] ** config['t'])
next_w = w - config['learning_rate'] * mt / (np.sqrt(vt) + config['epsilon'])
config['m'], config['v'] = m, v
```

Comments:

- **Correctness**: Functionally equivalent.
- **Style**: Similar to RMSProp, LLM updates dictionary directly. Variable names (`m_hat` vs `mt`) differ but are clear.

`deeplearning/classifiers/fc_net.py`

`TwoLayerNet`

Comments:

- The `TwoLayerNet` class was not implemented in the `hw2code` version provided (marked as `pass # TODO`). The `FullyConnectedNet` was implemented instead.

`FullyConnectedNet.__init__` (He Initialization)

LLM Implementation:

```
fan_in = dims[i]
std = np.sqrt(2.0 / fan_in)
self.params["W%d" % (i + 1)] = np.random.randn(dims[i], dims[i + 1]) * std
```

Staff Solution:

```
self.params['W%d' % (i + 1)] = np.random.randn(dims[i], dims[i + 1]) * math.sqrt(2. / d
```

Comments:

- **Correctness**: Identical logic.

- **Style**: LLM uses intermediate variables for clarity.

`FullyConnectedNet.__init__` (Zero Initialization)

LLM Implementation:

```
self.params["W%d" % (i + 1)] = np.zeros((dims[i], dims[i + 1]))
```

Staff Solution:

```
self.params['W%d' % (i + 1)] = np.random.normal(loc=0.0, scale=0, size=(dims[i], dims[i
```

Comments:

- **Correctness**: Identical result.
- **Style**: LLM's use of `np.zeros` is more idiomatic and efficient than `np.random.normal` with scale 0.

`FullyConnectedNet.loss` (Forward Pass)

LLM Implementation:

```
scores, caches, reg_loss = X, [], 0
for i in range(self.num_layers - 1):
    w, b = self.params['W%d' % (i + 1)], self.params['b%d' % (i + 1)]
    scores, cache = affine_relu_forward(scores, w, b)
    reg_loss += 0.5 * self.reg * np.sum(w ** 2)
    caches.append(cache)
w, b = self.params['W%d' % (self.num_layers)], self.params['b%d' % (self.num_layers)]
scores, cache = affine_forward(scores, w, b)
reg_loss += 0.5 * self.reg * np.sum(w ** 2)
caches.append(cache)
```

Staff Solution:

- Logic is identical.

  Comments:
- **Correctness**: Correctly handles arbitrary number of layers and regularization.

`FullyConnectedNet.loss` (Backward Pass)

LLM Implementation:

```
loss, dout = softmax_loss(scores, y)
loss += reg_loss

dout, dw, db = affine_backward(dout, caches.pop())
grads['W%d' % (self.num_layers)] = dw + self.reg * w
grads['b%d' % (self.num_layers)] = db

for i in range(self.num_layers - 2, -1, - 1):
    dout, dw, db = affine_relu_backward(dout, caches.pop())
    w = self.params['W%d' % (i + 1)]
    grads['W%d' % (i + 1)] = dw + self.reg * w
    grads['b%d' % (i + 1)] = db
```

Staff Solution:

- Logic is identical.

  Comments:
- **Correctness**: Correctly implements backpropagation for arbitrary depth.

`deeplearning/solver.py`

Store mean of l2 norm

LLM Implementation:

```
grad_norms = []
for p in self.model.params:
    grad_norms.append(np.linalg.norm(grads[p]))
self.log_grad_norm_history.append(np.mean(grad_norms))
```

Staff Solution:

```
self.log_grad_norm_history.append(np.mean([np.linalg.norm(dw, 2) for dw in grads.values
```

Comments:

- **Correctness**: Both are correct.
- **Style**: Staff solution is more concise (one-liner). LLM implementation is more verbose but readable.

`q_linearized_features.ipynb`

Backpropagation

LLM Implementation:

```
model.zero_grad()
y.backward()
```

Staff Solution:

```
model.zero_grad()
y.backward()
```

Comments:

- **Correctness**: Identical.

SVD Decomposition

LLM Implementation:

```
u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)
```

Staff Solution:

```
u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)
```

Comments:

- **Correctness**: Identical.

Principal Feature

LLM Implementation:

```
princple_feature = feature_matrix @ vh.T
```

Staff Solution:

```
princple_feature = feature_matrix @ vh.T
```

Comments:

- **Correctness**: Identical.

Two-layer Network

LLM Implementation:

- Defines network, adjusts biases ("elbows"), trains, and visualizes.
  Staff Solution:
- Defines network, trains, and visualizes.
  Comments:
- **Correctness**: LLM correctly adapted the code for a 2-layer network.
- **Completeness**: LLM included the bias initialization logic which ensures the "elbows" are in the correct range, which is good practice consistent with the 1-layer example.

`q_optimizer_init.ipynb`

Tune Batch Size

LLM Implementation:

```
batch_sizes = {
  'sgd_momentum': 100,
  'sgd': 400,
}
```

Staff Solution:

```
batch_sizes = {
  'sgd_momentum': 100,
  'sgd': 400,
}
```

Comments:

- **Correctness**: Identical values found.

Train Best FCNet

LLM Implementation:

```
lr = 1e-3
num_epochs = 10
batch_size = 100
lr_decay = 0.95
update_rule = 'adam'
```

Staff Solution:

```
lr = 5e-3
num_epochs = 10
batch_size = 100
lr_decay = 0.9
update_rule = 'adam'
```

Comments:

- **Correctness**: Both use Adam and similar epochs/batch size.
- **Differences**: Learning rate and decay differ slightly. This is expected as hyperparameter tuning is empirical.

q_rl.ipynb

gd_step

LLM Implementation:

```
dx, dy = grad_f(x, y)
new_x = x + eta * dx
new_y = y + eta * dy
```

Staff Solution:

```
dx, dy = grad_f(x, y)
new_x = x + dx * eta
new_y = y + dy * eta
```

Comments:

- **Correctness**: Identical logic.

`smoothed_f`

LLM Implementation:

```
tx = np.random.normal(x, sigma, size=(N,))
ty = np.random.normal(y, sigma, size=(N,))
return np.mean(f(tx, ty))
```

Staff Solution:

```
tx = np.random.normal(x, sigma, size=(N,))
ty = np.random.normal(y, sigma, size=(N,))
return np.mean(f(tx, ty), axis=0)
```

Comments:

- **Correctness**: Functionally equivalent. The staff solution explicitly specifies `axis=0`, which is good practice but not strictly necessary here as the input is 1D.

`finite_diff_grad_step`

LLM Implementation:

```
f_val = f(tx, ty)
f_val_dx = f(tx + delta, ty)
f_val_dy = f(tx, ty + delta)

grad_x = (np.mean(f_val_dx) - np.mean(f_val)) / delta
grad_y = (np.mean(f_val_dy) - np.mean(f_val)) / delta

new_x = x + eta * grad_x
new_y = y + eta * grad_y
```

Staff Solution:

```
# \hat{f}(x, y)
fxy = np.mean(f(tx, ty), axis=0)

# \hat{f}(x+delta, y)
fxdy = np.mean(f(tx + delta, ty), axis=0)

# \hat{f}(x, y+delta)
fxyd = np.mean(f(tx, ty + delta), axis=0)

# compute the gradient with respect to x and y
grad_x = (fxdy - fxy) / delta
grad_y = (fxyd - fxy) / delta

new_x = x + grad_x * eta
new_y = y + grad_y * eta
```

Comments:

- **Correctness**: Functionally equivalent. Both implementations correctly use the same random samples (`tx` , `ty`) for the perturbed evaluations, which is a crucial variance reduction technique.

`pg_grad_step`

LLM Implementation:

```
f_vals = f(tx, ty)
grad_x = np.mean((tx - x) * f_vals / (sigma**2))
grad_y = np.mean((ty - y) * f_vals / (sigma**2))
new_x = x + eta * grad_x
new_y = y + eta * grad_y
```

Staff Solution:

```
fxy = f(tx, ty)
grad_x = ((tx - x) * fxy / (sigma * sigma)).mean()
grad_y = ((ty - y) * fxy / (sigma * sigma)).mean()

new_x = x + grad_x * eta
new_y = y + grad_y * eta
```

Comments:

- **Correctness**: Identical logic.

`reparam_grad_step`

LLM Implementation:

```
dx, dy = grad_f(tx, ty)
grad_x = np.mean(dx)
grad_y = np.mean(dy)
new_x = x + eta * grad_x
new_y = y + eta * grad_y
```

Staff Solution:

```
grad_x, grad_y = grad_f(tx, ty)
grad_x = grad_x.mean()
grad_y = grad_y.mean()

new_x = x + grad_x * eta
new_y = y + grad_y * eta
```

Comments:

- **Correctness**: Identical logic.

`q_grad_step`

LLM Implementation:

```
f_vals = qf(tx, ty)
grad_x = np.mean((tx - x) * f_vals / (sigma**2))
grad_y = np.mean((ty - y) * f_vals / (sigma**2))
new_x = x + eta * grad_x
new_y = y + eta * grad_y
```

Staff Solution:

```
fxy = qf(tx, ty)
grad_x = ((tx - x) * fxy / (sigma * sigma)).mean()
grad_y = ((ty - y) * fxy / (sigma * sigma)).mean()

new_x = x + grad_x * eta
new_y = y + grad_y * eta
```

Comments:

- **Correctness**: Identical logic.

Loop for `q_grad_step`

LLM Implementation:

```
for i in range(num_its):
    x, y = q_grad_step(x, y, N, eta, sigma, delta)
    xs.append(x)
    ys.append(y)
```

Staff Solution:

```
for i in range(num_its):
    x, y = q_grad_step(x, y, N, eta, sigma, delta)
    xs.append(x)
    ys.append(y)
```

Comments:

- **Correctness**: Identical.